



UNIT 4

Computational Structures

Student Learning Outcomes

By the end of this chapter, students will be able to:

- Define and explain the purpose of primitive computational structures, including lists, stacks, queues, trees, and graphs.
- Identify and describe the characteristics and properties of different computational structures.
- Perform basic operations such as insertion, deletion, traversal, and searching on various computational structures.
- Understand and implement the LIFO (Last-In, First-Out) and FIFO (First-In, First-Out) principles in stacks and queues, respectively.
- Compare and contrast different types of trees and graphs, and apply appropriate operations to them.
- Analyze and choose the most suitable computational structure based on problem requirements, data organization, and performance considerations.
- Apply computational structures in real-world scenarios, including data organization, task scheduling, and network modeling.
- Combine different computational structures to solve complex problems and enhance functionality.

Introduction

In this chapter, we will explore key computational structures, such as lists, stacks, queues, trees, and graphs, which are fundamental in programming. We will examine their properties, operations, and how to implement them efficiently. Additionally, we will discuss selecting the appropriate structure based on specific problem requirements and demonstrate their application in real-world scenarios. By the end of this chapter, you will understand how to effectively use these structures in various contexts.

4.1 Primitive Computational Structures

In our daily lives, we often need to organize and manage data. For instance, think about the way you store your contacts on your phone, keep a to-do list, or even stack plates in your kitchen. Similarly, in the world of computers, we need to organize and manage data. Computational structures help us do that.

4.1.1 Lists

A list is a data structure used to store multiple pieces of data in a specific sequence. Each piece of data, known as an element, is positioned at a particular index within the list, facilitating easy access and management.

4.1.1.1 List Creation

In Python, Lists are created using square brackets '[]', with each item separated by a comma. Here's how you can create and work with a list in Python:

```
# Create a list of items
items= [ "Decorations" , "Snacks" , "cold drinks" , "Plates" , "Balloon"]
# Print the list
print(items)
```

In the above code:

"Items" is the name of the list.

The items inside the list are "Decorations", "Snacks", "Cold drinks", "Plates", and "Balloons".

Each item is enclosed in quotes (for text) and separated by a comma.

The print () function is used to display the list.

You can also add or remove items from the list. Lists are useful for storing multiple values in one variable and managing them efficiently.



A type of list known as a linked list uses nodes, where each node contains a value and a reference to the next node.

4.1.1.2 List Properties

List has following properties:

1. **Dynamic Size** A list in Python can change its size. You can add new items to the list or remove items without any problem. The list will automatically adjust to fit the changes.
2. **Index-Based Access** Every item in a list has a position, called an index. The first item has an index of 0, the second item has an index of 1, and so on. You can use these indexes to get specific items from the list.
3. **Ordered Collection** The order in which you add items to a list is preserved. This means that if you add an item first, it will stay in that position unless you change it.
4. **Heterogeneous Elements** A list can hold different types of items together. For example, you can have numbers, words, and even other lists all in one list.
5. **Mutability** Lists are changeable, meaning you can update, add, or remove items from the list after you create it.
6. **Supports Duplicates** A list can contain the same item more than once. This means you can repeat values in the list if needed.

```
# Example List
items = ["Decorations", "Snacks", "Cold drinks", "Plates", "Balloons"]
# Characteristics of Lists:
# Nature of the Data Structure
print("List of items:", items)
# Mutability
items.append("Party Hats")
items.remove("Snacks")
print("After modifications:", items)
# Homogeneous vs. Heterogeneous
print("Homogeneous List:", [1, 2, 3, 4])
print("Heterogeneous List:", ["Apple", 42, 3.14, True])
# Properties of Lists:
# Length
print("Number of items:", len(items))
# Indexing print("Item at index 2:", items[2])
# Ordering print("Order maintained:", items)
# Duplication
print("List with duplicates:", ["Decorations", "Snacks", "cold drinks", "Snacks"])
```

The code above shows different features of lists. First, it creates a list called 'items' with party essentials, keeping the items in the order they were added. We then demonstrate that lists can be changed by adding 'Party Hats' as a new item and removing 'Snacks', which shows that lists are dynamic.



Did you know that lists in Python can be nested, meaning you can have lists within lists? This feature is useful for representing complex data structures.

4.1.1.3 List Operations:

Let's explore some common operations we can perform on a list using real-world examples and Python code.

1. Insertion: Adding a new item to your list is like adding a new task to your to-do list. You can insert an item at different positions in the list. Here are several ways to do this:

a. Inserting at a Specific Position: You can insert an item at any position in the list using the 'insert()' function.

```
party_list = ["Buy drinks", "Buy decorations", "Buy snacks", "Buy cold drinks "]
party_list.insert(0, "Invite friends") # add Invite friends at start
print(party_list)
# Output: ["Buy drinks", "Buy decorations", "Buy snacks", "Buy cold drinks "]
```

b. Appending to the End: If you want to add an item to the end of the list, use the 'append()' function.

```
party_list = ["Invite friends", "Buy cold drinks", "Buy decorations", "Buy snacks",]
party_list.append("Order cake") # Adds 'Order cake' at the end
print(party_list)

# Output: ["Invite friends", "Buy cold drinks", "Buy decorations", "Buy snacks",
"Order cake"]
```

c. Extending the List: You can also add multiple items at once using the 'extend()' functions.

```
party_list = ["Invite friends", "Buy decorations ", "Buy snacks", "Buy cold
drinks"]
party_list.extend(["Party hats", "Streamers"]) # Add multiple items at once
print(party_list)
# Output: ["Invite friends", "Buy decorations", "Buy snacks ", "Buy cold
drinks", "Party hats", "Streamers"]
```



2. Deletion: Removing an item from your list is like crossing off a task you've completed. You can remove items in various ways

a. Removing by Value: Use the 'remove()' function to delete the first occurrence of a specific item.

```
party_list = ["Invite friends ", "Buy decorations", "Buy snacks", "Buy cold drinks"]
party_list.remove("Buy snacks") # Removes 'Buy snacks' from the list
print(party_list)
# Output: ['Invite friends', 'Buy decorations', 'Buy cold drinks']
```

b. Removing by Index: Use the 'pop()' function to remove an item at a specific index.

```
party_list = ["Invite friends ", "Buy decorations", "Buy cold drinks " ]
party_list.pop(0) # Removes the item at index 0
print(party_list)
# Output: ['Buy decorations', 'Buy cold drinks']
```

c. Removing All Items: Use the 'clear()' function to remove all items from the list.

```
party_list = ["Invite friends", "Buy decorations", "Buy cold drinks " ]
# Removes all items from the list
party_list.clear()
print(party_list) # Output : []
```

3. Searching: Finding an item in a list is similar to looking for a specific task in your to-do list. You can search for an item using different functions:

a. Checking for Existence: Use the 'in' keyword to check if an item exists in the list.

```
party_list = ["Invite friends", "Buy decorations", "Buy cold drinks"]
if "Buy cold drinks" in party_list:
    print("Buy cold drinks is on the list.") # Prints if 'Buy cold drinks' is found
else:
    print("Buy cold drinks is not on the list.")
# Output: Buy cold drinks is on the list.
```

- 
- b. **Finding the Index:** Use the 'index()' function to find the position of an item in the list.

```
party_list = ["Invite friends", "Buy decorations", "Buy cold drinks " ]  
index = party_list.index("Buy decorations") # Finds the position of 'Buy decorations'  
print("Index of 'Buy decorations':", index)  
# Output: Index of 'Buy decorations': 1
```

4.1.1.4 Applications of Lists

- **Data Storage and Manipulation:** Lists are commonly used to store and manage collections of data, such as records, entries, or values. They allow for easy insertion, deletion, and access to elements.
- **Stack and Queue Implementations:** Lists can be used to implement stack (LIFO) and queue (FIFO) data structures, which are fundamental for various algorithms and tasks in computing. Stacks and queues are discussed in sections 4.1.2 and 4.1.4, respectively.
- **Sorting and Searching Algorithms:** Lists are frequently used in algorithms for sorting and searching data. Algorithms like bubble sort, selection sort, and binary search often operate on lists to arrange or find data efficiently.
- **Function Arguments:** Lists are often used to pass multiple arguments to functions, enabling flexible and efficient handling of inputs in programming.
- **Matrix Operations:** Lists can be used to represent and manipulate matrices in numerical and scientific computing, where each list represents a row or column of the matrix.

4.1.2 Stacks

A stack is a simple data structure where you can only add or remove items from one end, known as the "top". Both insertion and deletion of elements occur at this top end. A stack operates on the Last-In, First-Out (LIFO) principle, meaning that the most recently added element is the first one to be removed.

4.1.2.1 Properties of Stack

- **LIFO Principle:** A stack works by the Last-In, First-Out rule. The last item added is the first one removed.
- **Single Access Point:** You can only add or remove items from the top of the stack.
- **Order Preservation:** The order in which you add items is preserved, with the last item added being the first to leave.
- **Size Flexibility:** A stack can either have a fixed size or grow as needed.

Sequential Access: You can only access items in the order they were added, starting from the top.



Figure 4.2: Stack of Books

4.1.2.1 Stack Operations:

There are two basic operations in a stack:

- **Push Operation:** Push means adding an item to the top of the stack. In Python, we use list data structure to implement the stack. Therefore, we use the list function `append()` to add an element to the top of the stack.
- **Pop Operation:** Pop means removing the item from the top of the stack. We use list `pop()` function to remove the element from the top of the stack.

Example: Imagine a stack of books shown in Figure 4.2, on a desk, you can only add a book on top of the stack or take the top book off. Here is a Python implementation of the book stack described above.

```
# Create an empty stack of books stack_of_books = []
print("Initial stack:", stack_of_books)      # Empty stack
# Add books to the stack (push operation)
print("\n Adding books to the stack (push operation):")
stack_of_books.append('Book A')
print("Stack after pushing 'Book A':", stack_of_books)
stack_of_books.append('Book B')
print("Stack after pushing 'Book B':", stack_of_books)
stack_of_books.append('Book C')
print("Stack after pushing 'Book C':", stack_of_books)
# Remove the top book from the stack (pop operation)
print("\nDeletion of top book (pop operation):")
top_book = stack_of_books.pop()
print("Removed book:", top_book)
print("Stack after popping the top book:", stack_of_books)
```

The code creates an empty stack to hold books. It then adds three books ("Book A", "Book B", and "Book C") one by one to the top of the stack. Finally, it removes the top book "Book C" from the stack, showing how the last book added is the first one taken off.



Did you know that stacks are used in algorithms like depth-first search (DFS) in graph traversal? They help manage the nodes to be explored in a LIFO manner.

4.1.3 Using Stacks in Computers

Stacks are used to evaluate expressions, function call management, balancing parentheses and undo mechanism. Let's discuss them one by one.

4.1.3.1 Infix, Postfix, and Prefix Notations

In mathematics and computer science, expressions can be written in different notations. The three main notations are infix, postfix, and prefix notations.

- **Infix Notation:** The operator is placed between the operands. This is the standard notation used in most arithmetic and algebraic expressions.

Example: $A + B$ or $(A + B) \times C$

- In $A + B$, the operator $+$ is between the operands A and B . Parentheses may be used to clarify the order of operations.
- **Postfix Notation (Reverse Polish Notation, RPN):** The operator is placed after the operands. This notation does not require parentheses to denote the order of operations.

Examples: $A + B = AB+$ and $(A + B) \times C = (AB+) \times C = AB + C \times$

- In $AB+$, you first encounter the operands A and B , followed by the operator $+$, indicating that A and B are to be added. In $AB + C \times$, you first add A and B , then multiply the result to C .
- **Prefix Notation (Polish Notation):** The operator is placed before the operands. Like postfix notation, prefix notation also does not require parentheses to indicate the order of operations. **Example:** $+AB$ or $+A \times BC$
- In $+AB$, the operator $+$ comes before the operands A and B , indicating that A and B are to be added. In $+A \times BC$, you first multiply B and C , then add A to the result.

Converting an Infix Expression to Postfix

Converting an infix expression (like $3 + 4 * 5$) to postfix notation (like $3 4 5 * +$) can make calculations easier for computers. We use a stack to help with this conversion. Here's a simple explanation:

1. Start with an empty stack and an empty output list.
2. Read the infix expression from left to right.

- 
- If you find an operand, add it directly to the output list.
 - If you find an operator (like +, -, *), push it onto the stack. But, if there is already an operator on the stack with higher or equal priority, pop it from the stack to the output list before pushing the new operator.
 - If you find a left parenthesis (, push it onto the stack.
 - If you find a right parenthesis), pop operators from the stack to the output list until you find a left parenthesis. Remove the left parenthesis from the stack but do not add it to the output list.

When you have processed the entire infix expression, pop any remaining operators from the stack to the output list.

Example:

Convert the infix expression $3 + 4 * 2$ to postfix notation.

- Start with an empty stack and output list.
- Read 3: Add to the output list: [3].
- Read +: Push to the stack: [+].
- Read 4: Add to the output list: [3, 4].
- Read *: Push to the stack (since * has higher priority than +): [+ , *].
- Read 2: Add to the output list: [3, 4, 2].
- Pop all operators from the stack to the output list: [3, 4, 2, *, +].

Result: The postfix expression is $3\ 4\ 2\ *\ +$.

4.1.3.2 Evaluating Postfix Expression

To solve the above $3\ 4\ 2\ *\ +$, postfix expressions we can use a stack. Here's a step-by-step guide: **Example:** Evaluate the postfix expression $3\ 4\ 5\ *\ +$ using a stack.

1. Start with an empty stack.
2. Read the expression from left to right.
3. When you find a number, push it onto the stack.
 - Push 3 onto the stack: [3]
 - Push 4 onto the stack: [3, 4]
 - Push 5 onto the stack: [3, 4, 5]
1. When you find an operator, pop the required number of items from the stack, perform the operation, and push the result back onto the stack.
 - Pop 5 and 4, multiply them: $4 * 5 = 20$
 - Push the result 20 back onto the stack: [3, 20]
 - Pop 20 and 3, add them: $3 + 20 = 23$
 - Push the result 23 back onto the stack: [23]
 - The final result on the stack is the answer to the expression.



4.1.3.3 Function Call Management:

Stacks are very useful in managing function calls in programming. Let's break it down with a simple example: Imagine you are making a series of phone calls. Each time you call someone, you write their name on a notepad. When you finish talking, you cross off their name.

How This Works:

1. **Making a Call:** When you make a call, you write down the name on top of the notepad. This is like pushing information onto the stack.
2. **Ending a Call:** After you finish talking, you cross off the name. This is like popping information off the stack.

In programming, when a function is called, its details (like what it needs to do and where to return after it's done) are written on the stack. This helps the computer remember what to do next. When the function finishes, it removes (or pops) its details from the stack, just like crossing off the name.

4.1.3.4 Balancing Parentheses:

When dealing with infix expressions, parentheses are used to show which operations should be done first. Balancing parentheses is important to ensure that every opening parenthesis has a matching closing parenthesis.

Steps to Check Parentheses:

1. Use a stack to keep track of opening parentheses.
2. When you find an opening parenthesis (, push it onto the stack.
3. When you find a closing parenthesis), pop the top of the stack.
4. If the stack is empty after processing all parentheses, they are balanced.
5. If the stack is not empty, or if you try to pop from an empty stack, the parentheses are not balanced.

Example: For the expression $(3 + 4) * (5 - 2)$, the parentheses are balanced. Each (has a matching).

4.1.3.5 Undo Mechanism:

Stacks are used to implement undo functionality in applications such as Not Pad and Microsoft Word. Each action you perform is saved on a stack. When you want to undo an action, the most recent action is removed from the stack and reversed. This allows you to go back to the previous state.

Example:

Imagine you are using a text editor and you make several changes to a document, like typing new text or deleting old text. Here's how the undo mechanism works:

1. **Performing Actions:** As you make changes, each action (such as typing or deleting text) is saved on a stack. For example, if you type "Hello", this action is

pushed onto the stack.

2. **Undoing an Action:** When you press the undo button, the most recent action (like typing "Hello") is removed from the stack. The text editor then reverses this action (e.g., it deletes "Hello" from the document).
3. **Multiple Undos:** If you continue to press undo, each previous action is reversed in the order they were performed. If you typed "World" after "Hello", pressing undo would first remove "World", then "Hello".

Class Activity

Consider the task of managing a stack of plates in a restaurant kitchen. Write a python code to simulate the operations of adding and removing plates from this stack.

Requirements:

1. Create a Stack Class:

- Define a Stack class using a list to represent the stack of plates.
- Implement methods for push (to add a plate), pop (to remove a plate), and peek (to view the top plate without removing it).

2. Demonstrate Stack Operations:

- Push the following plates onto the stack: Plate 1, Plate 2, Plate 3.
- Display the stack after each addition.
- Pop a plate from the stack, and print both the removed plate and stack.
- Peek at the top plate and print its value.

4.1.4 Queues

A queue is like a line in front of the a bank or a ticket counter. The first person to get in line is the first person to be served. In a computer, a queue works the same way. It keeps track of things so that the first item added is the first one to be taken out. Just like in a bank line, you add things to the back and remove them from the front, following the FIFO (First-In, First-Out) principle, as shown in Figure 4.3.

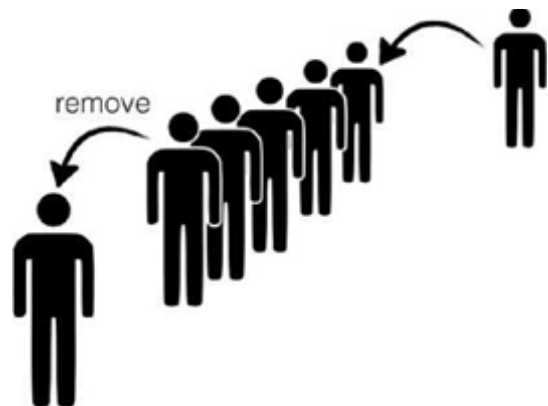


Figure 4.3: Queue of persons in front of the bank

4.1.4.1 Properties of Queue

Below are the key properties of a queue.

- **Order Preservation:** A queue keeps track of the order of items. The first item you put in the queue is the first one to come out.
- **FIFO Principle:** A queue works on the “First-In, First-Out” rule. This means the first item added to the queue is the first one to be removed.
- **Dynamic Size:** A queue can grow or shrink in size as you add or remove items. It adjusts automatically based on the number of items it holds.

4.1.4.2 Queue Operations

Queues support two primary operations:

- **Enqueue (Add an Item):** This is like adding a person to the end of the line. In a queue, you add items to the back.
- **Dequeue (Remove an Item):** This is like serving the person at the front of the line. In a queue, you take items out from the front.
- Additional operations might include checking if the queue is empty, retrieving the element at the front without removing it, and determining the size of the queue.

```
# Built-in module to implement queues in python
from queue import Queue
# Create a new queue
q = Queue ()
# Add people to the queue (Enqueue)
q.put (" Ahmed") # Adds Ahmed to the end of the queue
q.put ("Fatima") # Adds Fatima to the end of the queue
q.put ("Ali") # Adds Ali to the end of the queue
# View the person at the front of the queue (Peek)
front_person = q.queue [0] # Looks at the person at the front without
removing them
print(front_person) #Remove a person from the front of the queue
(Dequeue)
removed_person = q.getO # Removes and returns the person at the front
of the queue
print(removed_person)
# Add another person to the queue (Enqueue)
q.put("Sara") # Adds Sara to the end of the queue
# View the updated queue updated_queue = list(q.queue)
print(updated_queue)
```



The code manages a line of people using a queue. It adds people to the end of the line, checks who is at the front without removing them, and then serves (removes) the person at the front. Finally, it adds another person to the end and shows the updated line.

Tidbits

The 'peek' operation allows you to view the element at the front of the queue without removing it. It's similar to looking at the first book on the shelf to see which one will be picked next.

4.1.5 Using Queues in Computers

Queues are used to implement Breadth-First Search (BFS) in Graph, Task scheduling, Print Spooling while printing jobs and Call centre systems. Let's discuss them one by one.

4.1.5.1 Breadth-First Search Algorithm (BFS) in Graph

BFS is an algorithm used to explore and traverse data structures like trees or graphs. It systematically examines all the nodes at the present "level" or depth before moving on to nodes at the next level.

Imagine you and your friends are visiting a zoo. The zoo has many animal enclosures connected by paths. You want to see all the animals, so you need a plan to visit every enclosure without missing any. BFS is a method that helps you do this. To follow this method, you will use a queue, which is like a line of people waiting for their turn.

- 1. Start at the Entrance:** You begin your visit at the zoo entrance, which is the first enclosure you will explore. This entrance is like the starting point in a map, called the "start node".
- 2. Using a Queue to Visit Enclosures:** To stay organized, you put the entrance in a queue. The queue will help you visit each enclosure in the order you discover them. Think of the queue as a line where you add the enclosures you find so you can visit them one by one.
- 3. Visit the First Enclosure:** You start by visiting the first enclosure in the queue (the one at the front of the line). After seeing the animals there, you look at the map to find all the nearby enclosures connected to it. These nearby enclosures are called "neighboring enclosures." You then add (enqueue) these neighboring enclosures to the queue so you can visit them next.
- 4. Move to the Next Enclosure in the Queue:** Now, you move to the next enclosure in the queue and repeat the process. You visit the animals there, find its neighboring enclosures, and add them to the queue. This way, you visit all the enclosures step by step.

5. **Continue Until the Queue is Empty:** You keep following the queue until there are no more enclosures left in it. When the queue is empty, it means you have visited every enclosure in the zoo.

4.1.5.2 How We Traverse a Tree Using BFS ?

Breadth-First Search (BFS) is an algorithm used to explore and traverse tree structures by visiting nodes level by level. This approach uses a queue to manage the nodes that need to be visited. Below is a detailed explanation using a numeric tree example. The figure 4.4 illustrates the tree structure used in our BFS example:

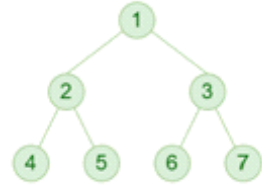


Figure 4.4: Balanced Binary Tree

BFS Process with Queue Operations:

1. **Start at the Root:**
 - Begin at the root node (1). This node is placed into the queue.
2. **Queue:** [1]
3. **Visit Node 1:**
 - Dequeue node 1 and visit it.
 - Add its children (nodes 2 and 3) to the queue.
4. **Queue after visiting 1:** [2, 3]
5. **Visit Node 2:**
 - Dequeue node 2 and visit it.
 - Add its children (nodes 4 and 5) to the queue.
6. **Queue after visiting 2:** [3, 4, 5]
7. **Visit Node 3:**
 - Dequeue node 3 and visit it.
 - Add its children (nodes 6 and 7) to the queue.
8. **Queue after visiting 3:** [4, 5, 6, 7]
9. **Visit Node 4:**
 - Dequeue node 4 and visit it.
 - Node 4 has no children, so the queue remains unchanged.
10. **Queue after visiting 4:** [5, 6, 7]
11. **Visit Node 5:**
 - Dequeue node 5 and visit it.
 - Node 5 has no children, so the queue remains unchanged.
12. **Queue after visiting 5:** [6, 7]
13. **Visit Node 6:**
 - Dequeue node 6 and visit it.
 - Node 6 has no children, so the queue remains unchanged.



14. Queue after visiting 6: [7]

15. Visit Node 7:

- Dequeue node 7 and visit it.
- Node 7 has no children, so the queue is now empty.

16. Queue after visiting 7: []

17. Completion:

- The queue is empty, indicating that all nodes have been visited in a level-by-level order.

4.1.5.3 Task Scheduling using a Queue

To understand task scheduling using a queue, let's compare it to a simple real-life scenario at a doctor's clinic.

- 1. Tasks as Patients:** Imagine that each task your computer needs to do is like a patient visiting a doctor. Just as patients arrive at the clinic and wait their turn to see the doctor, tasks arrive on your computer and wait to be processed.
- 2. Queue as the Waiting List:** The queue in a computer works like a waiting list at the clinic. It keeps track of all the tasks in the order they arrive, just like the list keeps track of the patients. This ensures that tasks are handled one by one in the correct order.
- 3. Enqueue Operation:** When a new task arrives (for example, when you press "Print" on your computer), it is added to the end of the queue, similar to how a new patient is added to the end of the waiting list. This process of adding a task to the queue is called *enqueue*.
- 4. Dequeue Operation:** As soon as the computer is ready to process a task, it removes the first task from the front of the queue, just like the doctor calling in the next patient from the waiting list. This process of removing a task from the queue for processing is called *dequeue*.
- 5. Why Use a Queue?** Using a queue ensures that tasks are processed in the order they arrive, without skipping any. This is similar to ensuring that each patient is seen by the doctor in the order they arrived. It helps the computer manage tasks efficiently and fairly.

4.1.5.4 Print Spooling

Print spooling is a system used to manage print jobs sent to a printer, ensuring that they are processed in the order they are received. This process involves the use of a queue to maintain organization and efficiency.

- **Managing Print Jobs:** When multiple documents are sent to a printer, they are placed in a queue, analogous to a line of people waiting at a service counter. Each document is added to the end of the queue, which maintains the order of arrival.

- 
- **Order of Printing:** The queue ensures that documents are printed sequentially, based on the order they were submitted. The document that was sent first will be printed first, while subsequent documents will wait their turn in the queue.
 - **Advantages of Using a Queue:** By employing a queue, print spooling prevents issues such as documents being printed out of sequence or confusion arising from simultaneous print requests. This orderly approach allows the printer to handle multiple tasks efficiently and fairly, maintaining an organized process.

4.1.5.5 Data Buffering

Queues play a crucial role in buffering data streams, which helps manage and process data efficiently. To understand this concept better, let's look at a couple of real-world examples.

1. **Online Video Streaming:** When you watch a video online, the video is not played all at once. Instead, it is buffered in small chunks. This means that as the video plays, a queue stores and loads parts of the video ahead of what you are currently watching. This buffering queue helps ensure that the video plays smoothly without interruptions, even if there are delays in data transfer from the internet.
2. **Printer Queue:** Imagine you have several documents to print, but your printer can only handle one document at a time. When you send multiple print jobs to the printer, they are placed in a queue. The printer then processes each document in the order they were received. This queue helps manage and organize the print jobs so that each document is printed one after the other without confusion.

4.1.6 Trees

A tree data structure organizes information in a way that spreads out from a main point called the root node. In a tree, each piece of information, called a node, can connect to other pieces, which are also nodes, forming a branching structure. This branching structure is different from a list, where items are organized one after the other in a straight line.

Example: In a family tree, the oldest ancestors represent the root node, serving as the starting point of the hierarchy. Each individual in the tree may have descendants, forming subsequent levels of the hierarchy, as illustrated in Figure 4.5. This hierarchical structure is not suitable for storage in a linear format, such as a list, due to the complex parent-child relationships. Therefore, a tree data structure is employed to efficiently store and access such hierarchical data, enabling clear representation and retrieval of information.



Figure 4.5: Family Tree

Certainly! Here's a simplified version focusing on the core properties of trees, using layman friendly language:



The concept of tree data structures was first introduced by computer scientist John McCarthy in 1960.

4.1.6.1 Properties of Trees

1. **Root Node:** The root is the very first or top node in a tree, like the main folder in a computer where all other folders and files are contained.
2. **Edges and Nodes:** Nodes are the individual elements in the tree, and they are connected by lines called edges. A node without any child nodes is called a leaf, similar to a file in a folder that doesn't contain any other files.
3. **Height:** The height of a tree is the longest path from the root node down to the farthest leaf. It tells us how deep or tall the tree is.
4. **Balanced Trees:** A tree is considered balanced if the branches on the left and right sides are nearly the same height. See the details in Section 4.1.7

Here's a simple example of how to create a family tree structure in Python using classes. This implementation shows how to create family members (nodes) and how to add children (descendants) to them.

```

class FamilyMember :
    def __init__(self, name): self.name = name self.children = []
class FamilyTree:
    def __init__(self, ancestor_name):
        self.root = FamilyMember(ancestor_name)
    def add_child(self, parent_name, child_name):
        parent_node = self._find_member(self.root, parent_name)
        if parent_node:
            parent_node.children.append(FamilyMember(child_name))
    def _find_member(self, current_member, name):
        if current_member.name == name:
            return current_member
        for child in current_member.children:
            result = self._find_member(child, name)
            if result:
                return result
        return None

```

- **Example usage**

```

family_tree = FamilyTree('Grandparent')
family_tree.add_child('Grandparent', 'Parent1')
family_tree.add_child('Grandparent', 'Parent2')
family_tree.add_child('Parent1', 'Child1')
family_tree.add_child('Parent1', 'Child2')

```

The above code shows:

- The 'FamilyMember' class represents a single person in the family tree. Each person has a name and can have children (descendants).
- The 'FamilyTree' class manages the entire tree, starting with an ancestor at the root. It includes functions to add children and find family members.

Therefore, the code generate the following tree in which each node (family member) is represented by a circle, and arrows indicate the parent-child relationships as shown in

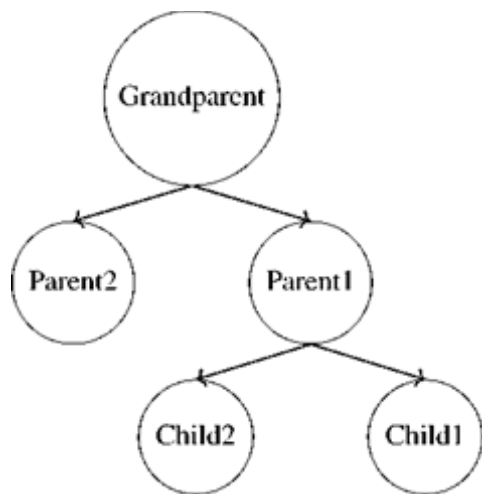


Figure 4.6.

4.1.7 Balanced Tree

A Balanced Tree is a type of tree data structure where the height of the tree is kept as small as possible by ensuring that the tree remains balanced. This balance is achieved by maintaining that the height difference between the left and right subtrees of any node is no more than a certain value



Figure 4.7: Balanced Tree structure for organizing books in a Store

(commonly 1). Balanced trees are essential for efficient operations such as searching, insertion, and deletion, as they keep these operations close to $O(\log n)$ time complexity.

Example: Organizing Books in a Library

Consider a library that needs to efficiently organize and locate books on shelves. If the books were simply arranged in a linear order on a single shelf, finding a specific book might require scanning through many books. Instead, imagine the books are organized using a balanced tree structure.

At the root of this tree, we have a central category, such as **Science**. This category could then be divided into subcategories like **Physics** and **Biology**, each forming the children of the root. Within each subcategory, further divisions can be made, such as separating **Physics** into **Classical Mechanics** and **Quantum Physics**, and **Biology** into **Botany** and **Zoology**. This process continues, ensuring that no single branch of the tree becomes significantly longer than the others.

In this balanced tree, searching for a book on **Quantum Physics** involves traversing down the branches of the tree: first to **Science**, then to **Physics**, and finally to **Quantum Physics**. The balanced nature of the tree ensures that this search is quick, as the tree's height remains logarithmic relative to the number of categories, avoiding lengthy search times.

This concept is illustrated in Figure 4.7 while the computer science tree has upside down as shown in Figure 4.8.

4.1.7.1 Operations on Tree

In a tree structure, there are some basic operations that help us manage and work with the data. Let's break them down:

- **Insertion:** Imagine you're adding a new family member to your family tree. Insertion is like finding the right spot in the tree and adding the new member there. We connect the new person to their parent in the tree, making sure they are part of the family structure.

Here's how you can insert a new member into the Family Tree:

- **Deletion:** Sometimes, we need to remove a member from the tree. Deletion is the process of taking someone out of the family tree. We carefully remove them, making sure the rest of the tree still makes sense and stays connected.
- **Traversal:** Tree traversal like walking through the family tree to visit each member. There are different ways to do this, like starting from the top and visiting each person one by one, or visiting all the children of a person before moving to the next branch. There are three common types of tree traversals are there: In-order, Pre-order, and Post-order. We will use the following simple tree structure with numeric data to understand traversal functions more easily.

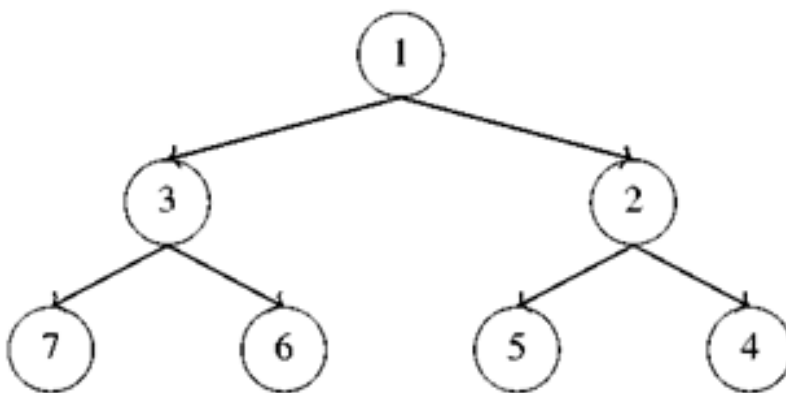


Figure 4.8: Tree Structure with Numeric Data



This tree has a root node with the value **1**. It has two child nodes, **2** and **3**, which themselves have children as shown in Figure 4.8.

1. In-order Traversal

In-order traversal means visiting the left subtree first, then the root node, and finally the right subtree.

Example: The in-order traversal of the above tree is given below.

1. Visit node 4 (leftmost node of 2)
2. Visit node 2
3. Visit node 5 (right child of 2)
4. Visit node 1 (root)
5. Visit node 6 (leftmost node of 3)
6. Visit node 3
7. Visit node 7 (right child of 3)

The in-order traversal output is: 4, 2, 5, 1, 6, 3, 7.

2. Pre-order Traversal

Pre-order traversal involves visiting the root node first, then the left subtree, followed by the right subtree.

Example: The pre-order traversal of the above tree is given below:

1. Visit node 1 (root)
2. Visit node 2
3. Visit node 4 (leftmost node of 2)
4. Visit node 5 (right child of 2)
5. Visit node 3
6. Visit node 6 (leftmost node of 3)
7. Visit node 7 (right child of 3)

The pre-order traversal output is: 1, 2, 4, 5, 3, 6, 7.

3. Post-order Traversal

Post-order traversal means visiting the left subtree first, then the right subtree, and finally the root node. For our

Example: The post-order traversal of the above tree is given below:

1. Visit node 4 (leftmost node of 2)
2. Visit node 5 (right child of 2)
3. Visit node 2
4. Visit node 6 (leftmost node of 3)
5. Visit node 7 (right child of 3)
6. Visit node 3
7. Visit node 1 (root)

The post-order traversal output is: 4, 5, 2, 6, 7, 3, 1.

Searching: Searching is like looking for a specific member in the family tree. We go through the tree, checking each person until we find the one we're looking for. This helps us quickly locate information in the tree.

4.1.7.2 Types of Trees:

Trees come in various types, each with its own unique structure and purpose. There are two common types of trees: Binary Trees and AVL Trees. These types help in organizing and managing data in efficient ways.

1. Binary Tree:

A binary tree is a hierarchical data structure in which each node can have at most two children, known as the left and right children as shown in Figure 4.9. This structure is analogous to a family tree, where each individual may have up to two descendants, ensuring a well-organized and balanced arrangement of data for efficient management and retrieval.

Example: In the binary tree example shown in figure 4.9:

1. The "Grandparent" is the root, and it has two children, "Parent1" and "Parent2".
2. Each parent can have their own children (like "Child1", "Child2". etc.).

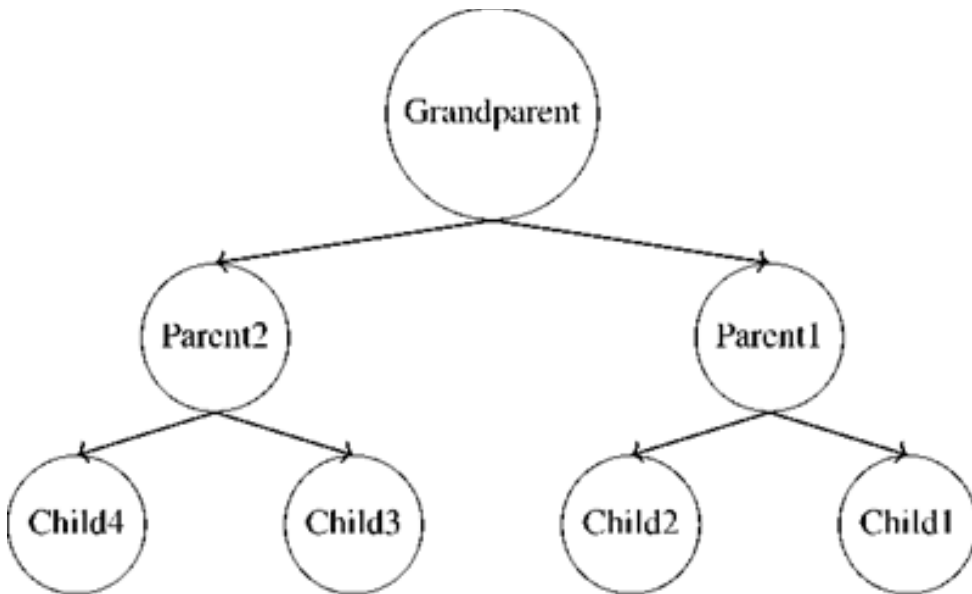


Figure 4.9: Binary Tree Representation of a Family Tree

2. AVL Trees:

An AVL tree is a type of binary tree that keeps itself balanced, meaning the height of the tree is managed carefully. If one side of the tree starts getting too tall compared to the other, the tree rearranges itself to maintain balance.

Example: Imagine you are organizing books on shelves. You want to make sure that the shelves are balanced, so one side isn't heavier than the other. This is similar to an AVL tree as shown in Figure 4.10.

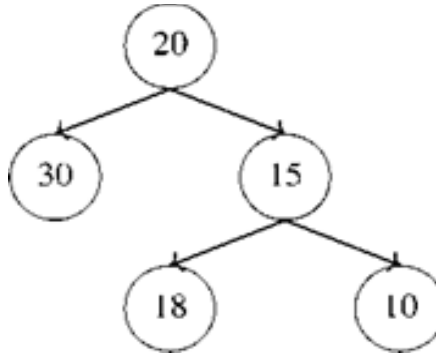


Figure 4.10: AVL Tree.

In the above example:

- The tree stays balanced after each new number (like a book on a shelf) is added.
- The height difference between the left side (with nodes 15, 10, 18) and the right side (with node 30) remains minimal, keeping the tree balanced.

4.1.7.2 Applications of Trees

1. **File Systems:** Pre-order tree traversal is useful for creating backups of file systems. By visiting the root first and then recursively backing up each directory, it ensures that directories are backed up before their contents.
2. **File System Deletion:** In file systems, Post-order traversal ensures that files and directories are deleted in the correct order—by first deleting all sub directories and files before deleting the parent directory.
3. **Hierarchical Data Representation:** Trees are used in representing data with a clear hierarchical relationship, such as organisational charts and family trees.
4. **Decision Making:** Trees, such as decision trees, are used in algorithms to make decisions based on various conditions and outcomes.
5. **Syntax Parsing:** In programming languages, trees are used to parse and represent the syntax of source code and help compilers to understand and process code structure.



4.1.8 Introduction to Graphs

A Graph is a data structure that consists of a set of **vertices** (or nodes) connected by **edges**. Graphs are used to represent networks of connections, where each connection is a relationship between two vertices. These vertices can represent anything, like cities, people, or even abstract concepts, and the edges represent the relationships or pathways between them.

Imagine you are mapping out all the cities in Pakistan and the roads that connect them. Each city is a vertex, and each road between two cities is an edge. Unlike a tree, a graph does not have a single "root" and does not follow a hierarchical structure. In a graph, any two vertices can be connected, creating a complex web of relationships.

Example: In a social network, each person can be connected to many others, forming a graph. There is no single starting point, and people (vertices) can have multiple connections (edges) that do not follow a strict parent-child relationship like in a tree.

Difference from a Tree: While both graphs and trees are used to represent relationships between objects, a tree is a special kind of graph with some important differences:

- A **Tree** is hierarchical, meaning it has a single root node from which all other nodes branch out. In contrast, a **Graph** does not necessarily have a hierarchy or a root.
- In a **Tree**, there is exactly one path between any two nodes, ensuring no cycles (loops). However, in a **Graph**, there can be multiple paths between nodes, and cycles are allowed.
- **Trees** are often used to represent structured data like family trees or organizational charts. **Graphs** are more flexible and can represent a broader range of connections, such as networks, web links, or transport systems.

4.1.8.1 Characteristics of Graphs

Graphs have several defining features that help us understand and use them effectively:

- **Vertices (Nodes):** These are the individual points or entities in a graph. For example, in a social network graph, each user is represented by a vertex.
- **Edges (Links):** These are the connections between vertices. For example, in a transport system graph, each road connecting two cities is an edge.

4.1.8.2 Properties of Graphs

Graphs also have specific details that describe their structure:

- **Degree:** This is the number of edges connected to a vertex. For instance, if a city is connected to three other cities, the degree of that city's vertex is 3.
- **Weight:** In some graphs, edges have weights that represent values like distances or costs. For example, if a road between two cities is 50 kilometres long, its edge might have a weight of 50.

- 
- **Direction:** Edges can be either directed or undirected. Directed edges have a one-way connection, meaning a road from city A to city B does not necessarily have a return road from B to A. Undirected edges represent a two-way connection.

4.1.8.3 Operations on Graphs

Graphs support various operations to manipulate and explore their structure. Here are some essential operations:

- **Insertion:** Adding a new vertex or edge to the graph.
- **Example:** If a new city is added to the transport map, it introduces a new vertex and possibly new edges.
- **Deletion:** Removing a vertex or edge from the graph.
- **Example:** If a city is removed from the map, its corresponding vertex and all edges connected to it are deleted.
- **Traversal:** Visiting all vertices and edges systematically. Traversal can be done using different functions:
- **Breadth-First Search (BFS):** This technique starts from a source vertex and explores all its immediate neighbors before moving to the next level of vertices.
Example: Imagine you are at a central train station in a city, and you want to visit all the nearby neighborhoods. First, you would take the trains that directly connect to the central station and explore all the neighborhoods they reach. Once you have visited all these neighboring areas, you then move to the next set of neighborhoods that are directly connected to the previously visited ones. This process continues until you have explored all reachable neighborhoods from the central station.
 - **Depth-First Search (DFS):** This technique starts from a source vertex and explores as far as possible along each branch before backtracking.
 - **Example:** Imagine you are exploring a multi-storey building. You start at the entrance and decide to explore each floor completely before moving to the next floor. You walk down each hallway and check every room on the current floor before going up or down to the next floor. If you reach the end of a hallway, you go back to explore any other rooms you might have missed on that floor before moving to the next one.
 - **Searching:** Finding a specific vertex or edge in the graph. Searching operations help locate particular cities or routes efficiently.

4.1.8.4 Types of Graphs

Graphs can be classified into several types based on their structure and properties. The main types of graphs are directed, undirected, and weighted. Each type has its own characteristics, which can be better understood through simple examples.

- **Directed Graphs:** In a directed graph, edges have a direction, which means they go from one vertex to another in a specific way as shown in Figure 4.11.

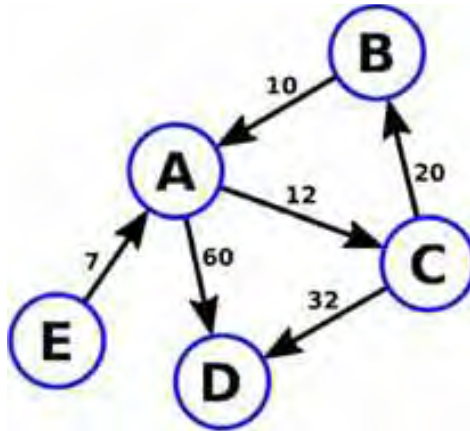


Figure 4.11: Directed Weighted Graph

Example: Consider a graph shown in figure 4.11 . If you want to travel from city A to city B, you can only go in the direction permitted by the city's sign. If there's no one-way street going from city A to city B, you cannot travel directly from city A to B.

- **Undirected Graphs:** In an undirected graph, edges do not have a direction. This means that if there is a connection between two vertices, you can travel in both directions.

Example: Consider a graph that shown in Figure 4.12, if Person A is friends with Person B, then Person B is also friends with Person A. There is no restriction on the direction of the friendship, so you can move freely between friends.

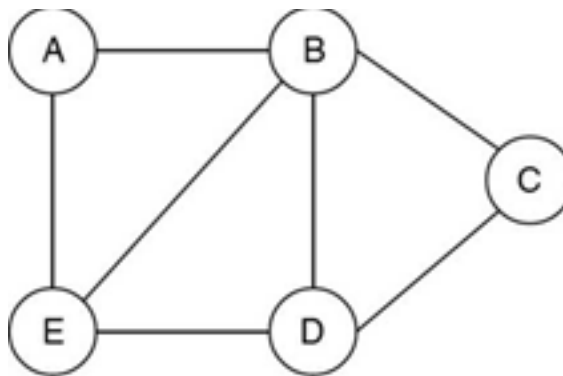


Figure 4.12: Undirected Graph

- 
- **Weighted Graphs:** In a weighted graph, each edge has a weight or cost associated with it. This weight represents the distance, time, or cost required to travel from one vertex to another as shown in Figure 4.11.

Example: Imagine a map of a city where each road has a different distance or travel time. If you want to travel from one landmark to another, the map provides the distance or travel time for each road. This information helps you determine the shortest or quickest route between landmarks.

Graph Implementation Using Python

Graphs can be implemented in Python using libraries such as NetworkX. Here's a basic example:

```
import networkx as nx
import matplotlib.pyplot as plt
# Create a graph object
G = nx.Graph()
# Add vertices (nodes)
G.add_node('A')
G.add_node('B')
G.add_node('C')
# Add edges (links) between vertices
G.add_edge('A', 'B')
G.add_edge('B', 'C')
# Draw the graph
nx.draw(G, with_labels=True)
plt.show()
```

In the above code:

- We first create a graph object.
- Nodes 'A', 'B', and 'C' are added to the graph.
- Edges are added to connect these nodes.
- The graph is drawn and displayed using Matplotlib.

This example demonstrates a simple graph with three nodes and two edges, visualizing how vertices and edges are represented programmatically.

4.2 Choosing the Right Data Structure for the Problem

When solving problems in computer science and programming, selecting the right data structure is crucial. Different data structures are suited to different types of problems, and understanding when to use each one can make your solutions more efficient and



easier to implement. Let's break down when each of the five data structures: Stack, Queue, List, Tree, and Graph, is most appropriate.

4.2.1 Stack: Best for Last-In, First-Out (LIFO) Problems

When to Use: A stack is ideal when you need to process data in the reverse order of how it was received. This is known as 'Last-In, First-Out' (LIFO). Stacks are often used in scenarios where you need to backtrack, like in navigating through web pages (where you can go back to the previous page) or evaluating mathematical expressions. Common applications include:

- **Expression Evaluation:** Converting infix expressions to postfix or prefix, and evaluating them.
- **Backtracking Algorithms:** Such as solving puzzles like mazes, or the Depth-First Search (DFS) in graphs.
- **Undo Mechanisms:** In software like text editors, where you can undo the most recent action.

4.2.2 Queue: Best for First-In, First-Out (FIFO) Problems

When to Use: A queue is perfect for scenarios where the order of processing needs to follow the 'First-In, First-Out' (FIFO) principle. This means the first element added to the queue will be the first one to be removed. Queues are commonly used in:

- **Breadth-First Search (BFS):** In graphs or trees, where you explore all nodes at the current level before moving to the next.
- **Task Scheduling:** Where tasks are processed in the order they arrive, like print job management or handling requests in a web server.
- **Simulation of Real-World Scenarios:** Such as modeling a line of customers in a store.

4.2.3 List: Best for Ordered Collections

When to Use: Lists are versatile and useful when you need to store an ordered collection of elements. They are especially appropriate when the order of elements matters, but direct access to elements by index is also needed. Use lists in:

- **Dynamic Arrays:** Where you need to frequently add or remove elements.
- **Storing Sequences:** Such as the order of elements in a to-do list or playlist.
- **Implementing Other Data Structures:** Lists can be used as the building blocks for stacks, queues, and more.

4.2.4 Tree: Best for Hierarchical Data

When to Use: Trees are essential when dealing with hierarchical data, where elements are naturally organized in a parent-child relationship. Trees are used in:

- **Hierarchical Databases:** Representing organizational structures, file systems, or XML/HTML documents.

- 
- **Binary Search Trees (BSTs):** For efficient searching, insertion, and deletion operations.
 - **Understanding Code Structure:** When building compilers, trees are used to break down and understand the code's structure, helping the computer figure out its step-by-step instructions.

4.2.5 Graph: Best for Networked Data

When to Use: Graphs are used when you need to represent a set of objects (nodes) and the connections (edges) between them. Graphs are particularly useful in problems involving networks, such as:

- **Social Networks:** Representing users and their connections (friends, followers).
- **Pathfinding Algorithms:** Like finding the shortest path in maps or game development.
- **Dependency Resolution:** Such as determining the build order of software projects with interdependent components.

4.3 Combining Computational Structures for Complex Problems

Combining different data structures, such as lists, stacks, queues, trees, and graphs, is essential for solving complex problems effectively. Each of these structures has its own strengths, and when we combine them, we can tackle more challenging problems with greater success.

This combination allows us to create stronger and more efficient solutions. It helps us manage complex data relationships and improve our ability to solve problems. By using these structures together, we can develop solutions that are better suited to specific needs and challenges, ultimately enhancing both performance and functionality.

4.3.1 Combining Stacks and Queues to Evaluate Expressions

Stacks and queues are powerful tools in evaluating mathematical expressions, especially when the expressions involve complex operations. By combining these two data structures, we can efficiently manage the order of operations, handle parentheses, and ensure the correct evaluation of expressions.

4.3.2 Combining Stack and Graph to perform Depth-First Search (DFS)

Depth-First Search (DFS) is a fundamental algorithm used to explore or traverse through a graph or tree structure. It uses a stack data structure to keep track of the vertices or nodes that need to be visited. This method is particularly useful when we need to explore as far as possible along a branch before backtracking, making it ideal for situations

where deep exploration is required.

Example: Consider the graph shown in Figure 4.13, as a tree with numeric nodes:

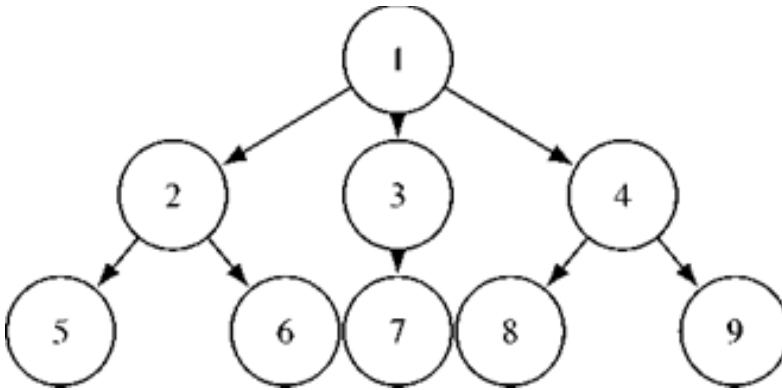


Figure 4.13: Graph Representation of a Tree with Numeric Nodes

In this graph:

- Node 1 is the root.
- Nodes 2, 3, and 4 are children of node 1.
- Nodes 5 and 6 are children of node 2.
- Node 7 is a child of node 3.
- Nodes 8 and 9 are children of node 4.

Here's how the DFS algorithm uses a stack to traverse this graph:

1. Start with an empty stack and push the root node (1) onto the stack.
2. **Step 1:** Pop node 1 from the stack and mark it as visited. Push all of its unvisited children (nodes 2, 3, 4) onto the stack. The stack now contains nodes [4, 3, 2].
3. **Step 2:** Pop the top of the stack (node 2). Mark node 2 as visited and push its unvisited children (nodes 5, 6) onto the stack. The stack now contains nodes [4, 3, 6, 5].
4. **Step 3:** Pop the top of the stack (node 5). Mark node 5 as visited. Since node 5 has no children, nothing is added to the stack. The stack now contains nodes [4, 3, 6].
5. **Step 4:** Pop the top of the stack (node 6). Mark node 6 as visited. Since node 6 has no children, nothing is added to the stack. The stack now contains nodes [4, 3].
6. **Step 5:** Pop the top of the stack (node 3). Mark node 3 as visited and push its unvisited child (node 7) onto the stack. The stack now contains nodes [4, 7].
7. **Step 6:** Pop the top of the stack (node 7). Mark node 7 as visited. Since node



7 has no children, nothing is added to the stack. The stack now contains node [4].

8. **Step 7:** Pop the top of the stack (node 4). Mark node 4 as visited and push its unvisited children (nodes 8, 9) onto the stack. The stack now contains nodes [9, 8].

9. **Step 8:** Pop the top of the stack (node 8). Mark node 8 as visited. Since node 8 has no children, nothing is added to the stack. The stack now contains node [9].

10. **Step 9:** Pop the top of the stack (node 9). Mark node 9 as visited. Since node 9 has no children, the stack is now empty.

The traversal order of the nodes using DFS in this example would be: **1, 2, 5, 6, 3, 7, 4, 8, 9.**

In this example, the DFS algorithm starts at the root node (1) and explores as far as possible along each branch before backtracking. By using a stack, the algorithm traverses the graph depth-first, visiting nodes in the order: **1, 2, 5, 6, 3, 7, 4, 8, 9.** The stack helps manage the nodes to visit next, ensuring a complete exploration of each branch before moving to the next.

4.3.3 Combining Queue and Graph to Perform Breadth-First Search (BFS)

Breadth-First Search (BFS) is another fundamental algorithm used to explore or traverse through a graph or tree structure. Unlike Depth-First Search, BFS uses a queue data structure to keep track of the vertices or nodes that need to be visited. This method is particularly useful when we need to explore all nodes at the present depth level before moving on to nodes at the next depth level, making it ideal for situations where we need to find the shortest path or explore all possible paths level by level.

Example: Consider the same graph represented in Figure 4.13, as a tree with numeric nodes:

Here's how the BFS algorithm uses a queue to traverse this graph:

1. Start with an empty queue and enqueue the root node (1).
2. **Step 1:** Dequeue node 1 and mark it as visited. Enqueue all of its unvisited children (nodes 2, 3, 4). The queue now contains nodes [2, 3, 4],
3. **Step 2:** Dequeue the front of the queue (node 2). Mark node 2 as visited and enqueue its unvisited children (nodes 5, 6). The queue now contains nodes [3, 4, 5, 6],
4. **Step 3:** Dequeue the front of the queue (node 3). Mark node 3 as visited and enqueue its unvisited child (node 7). The queue now contains nodes [4, 5, 6, 7],
5. **Step 4:** Dequeue the front of the queue (node 4). Mark node 4 as visited and enqueue its unvisited children (nodes 8, 9). The queue now contains nodes [5, 6,



7, 8, 9],

6. **Step 5:** Dequeue the front of the queue (node 5). Mark node 5 as visited. Since node 5 has no children, nothing is added to the queue. The queue now contains nodes [6, 7, 8, 9],
7. **Step 6:** Dequeue the front of the queue (node 6). Mark node 6 as visited. Since node 6 has no children, nothing is added to the queue. The queue now contains nodes [7, 8, 9],
8. **Step 7:** Dequeue the front of the queue (node 7). Mark node 7 as visited. Since node 7 has no children, nothing is added to the queue. The queue now contains nodes [8, 9].
9. **Step 8:** Dequeue the front of the queue (node 8). Mark node 8 as visited. Since node 8 has no children, nothing is added to the queue. The queue now contains node [9].
10. **Step 9:** Dequeue the front of the queue (node 9). Mark node 9 as visited. Since node 9 has no children, the queue is now empty.

The traversal order of the nodes using BFS in this example would be: **1, 2, 3, 4, 5, 6, 7, 8, 9**. In this example, the BFS algorithm, using a queue, guarantees that all nodes at the current level of the tree or graph are explored before proceeding to the next level. This approach is particularly effective for finding the shortest path in unweighted graphs or exploring all possible connections at each level of depth.

4.3.4 Combining List and Stack to Perform Sequential Operations

Lists and stacks are fundamental data structures that, when combined, provide a powerful way to manage and process data in a controlled sequence. A list allows for flexible data storage and access, while a stack imposes a Last-In-First-Out (LIFO) order, making it useful for tasks where the most recent element needs to be processed first.

Example: Consider a scenario where we need to reverse the order of a list of numbers using a stack. Let's take the following list of numbers as an example:

Original List: [10,20,30,40,50]

To reverse the order of this list using a stack, we can follow these steps:

1. Initialize an empty stack.
2. **Step 1 :** Push each element of the list onto the stack. After pushing all elements, the stack will contain:
Stack: [10 20 30 40 50]
3. **Step 2:** Pop elements from the stack one by one and insert them back into the list. Since the stack follows the LIFO order, the elements will be inserted in reverse order.
4. **Step 3:** After all elements are popped and inserted back into the list, the list



will now be:

Reversed List: [50,40,30,20,10]

In this example, the stack is used to temporarily hold the elements of the list, enabling us to reverse the order by taking advantage of the LIFO nature of the stack. The combination of a list and a stack allows us to efficiently perform operations that require reversing, undoing, or backtracking, which are common in various algorithms and applications.

Summary

This chapter offers a comprehensive overview of fundamental computational structures, including lists, stacks, queues, trees, and graphs. It details their properties, operations, and applications, emphasizing their roles in organizing and managing data. The chapter also addresses the criteria for selecting appropriate data structures based on problem requirements. Additionally, it explores the integration of different computational structures to address complex problems effectively. By providing a detailed understanding of these structures, the chapter aims to enhance the reader's ability to apply them effectively in various computational contexts.

EXERCISE

Multiple Choice Questions

1. Which function is used to add an item at the end of the list in python?
 - a) insert()
 - b) append()
 - c) remove()
 - d) pop()
2. What does the 'in' keyword do when used with python list?
 - a) Adds an item to the list.
 - b) Removes an item from the list.
 - c) Checks if an item exists in the list.
 - d) Returns the length of the list.
3. Which operation removes an item from the top of the stack?
 - a) Push
 - b) Pop
 - c) Peek
 - d) Add

- 
4. When converting an infix expression to postfix notation, what does a stack help to manage?
- a) Order of operands
 - b) Priority of operators
 - c) Parentheses
 - d) Both b and c
5. Which operation is used to add an item to a queue?
- a) Dequeue
 - b) Peek
 - c) Enqueue
 - d) Remove
6. In the context of Breadth-First Search (BFS) in a tree, which operation of queue is used to visit nodes level by level?
- a) Adding nodes to the end of a stack.
 - b) Removing nodes from the end of a list.
 - c) Enqueueing nodes to a queue.
 - d) Dequeueing nodes from a stack.
7. Which of the following tree traversals visits the root node before its children?
- a) In-order
 - b) Pre-order
 - c) Post-order
 - d) Level-order
8. Which of the following is true about the height of a tree?
- a) The height is the number of edges from the root to the deepest node
 - b) The height is the number of nodes from the root to the deepest node
 - c) The height is the number of children of the root node
 - d) The height is always equal to the number of nodes in the tree
9. Which graph traversal explores all immediate neighbors of a vertex before moving to the next level?
- a) Depth-First Search (DFS)
 - b) Breadth-First Search (BFS)
 - c) Depth-Limited Search (DLS)
 - d) Iterative Deepening Search (IDS)
10. For which scenario would a graph data structure be most appropriate?
- a) Managing a to-do list
 - b) Modeling a line of customers in a store
 - c) Representing connections in a social network



Short Questions

1. Explain how the 'extend()' function works in python lists. Provide an example.
2. Explain the potential issues which could arise when two variables reference the same list in a program? Provide an example.
3. Define a stack and explain the Last-In, First-Out (LIFO) principle.
4. How does the stack help in balancing parentheses in an expression? Describe the process.
5. Differentiate between the Enqueue and Dequeue operations of queue.
6. Explain how the concept of a queue is applied in a computer's job scheduling system?
7. How can lists be utilized to implement other data structures?
8. Differentiate between the Depth-First Search (DFS) and Breadth-First Search (BFS).

Long Questions

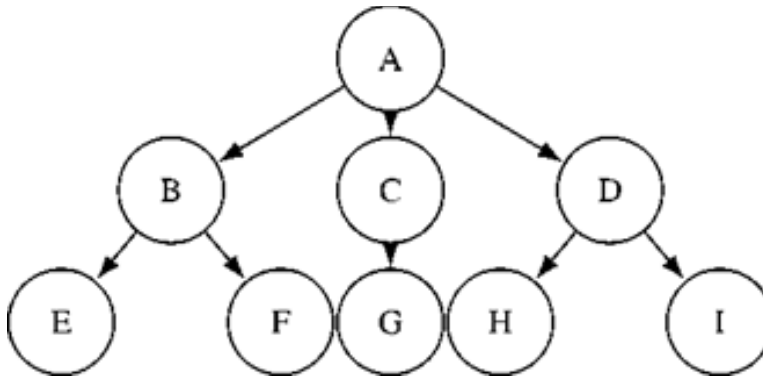
1. Discuss the dynamic size property of lists in Python. How does this property make lists more flexible?
2. Write a python program to manage a project's task list. Start with an initial list containing the following tasks=[design, development, testing, deployment]. Perform the following operations on the tasks list:
 - Add 'review' to the end of the list.
 - Insert 'planning' at the third position in the list.
 - Update 'testing' to quality assurance.
 - Remove 'development ' from the list by its value.
 - Remove the task at index 1 from the list.
 - Find the index of the task 'deployment ' and print it.
 - Print the final list after all modifications.
 - Count and print the number of tasks in the list.
3. Discuss two real-world situations where stacks are commonly used. Explain how stack operations function in these contexts.
4. Consider the following infix expression $(5 + 2) * (3 - 1)$.
 - (a) Describe the step-by-step process to convert this infix expression into postfix notation using a stack.
 - (b) Illustrate the state of the stack and the output list after processing each token of the infix expression.

Hint: A token is a single element in an expression, such as a number, operator, or parenthesis.
5. Write a Python code to simulate a queue of print jobs. The code perform

following operations:

- (a) Create an empty queue.
 - (b) Add three print jobs to the queue.
 - (c) Display the print job at the front of the queue without removing it.
 - (d) Remove the print job from the front of the queue.
 - (e) Add another print job to the queue.
 - (f) Display the updated queue of print jobs.
6. Explain the different types of tree traversals (In-order, Pre-order, Post-order). Provide an example of each traversal using a given tree structure, and explain how each traversal method processes the nodes.

Consider the following graph and perform the operations specified below:



Required Operations:

- (a) Start from node A, write down the sequence of nodes visited during a depth-first search traversal of the graph.
 - (b) Start from node A, write down the sequence of nodes visited throughout a breadth-first search traversal of the graph.
8. Consider the task of organizing a desk. Describe how you can use both a stack and a queue to manage tasks such as sorting papers, putting away books, and cleaning the desk.